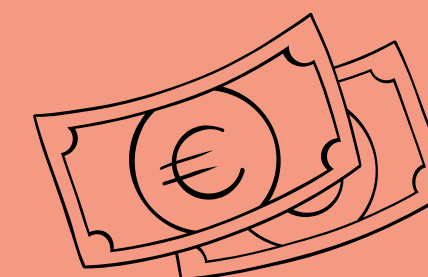




DÉtection automatique de FAUX billets en EUROS

-Mission ONCFM



Sommaire

1. Contexte & Objectifs
2. Les données
3. Prétraitement des données
4. Analyse exploratoire (EDA)
5. Les 4 algorithmes testés
6. Comparaison des performances
7. Optimisation & Modèle final
8. L'application de prédiction
9. Démonstration en direct



Contexte & Objectifs

Objectif :

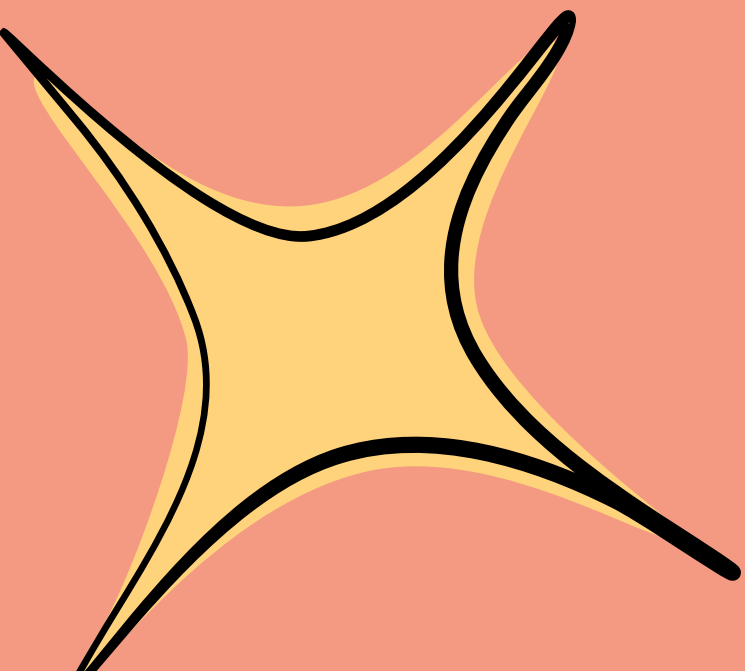
Construire un algorithme capable de classer automatiquement un billet (vrai / faux) à partir de 6 mesures géométriques.

Problème :

Les différences géométriques entre vrais et faux billets sont imperceptibles à l'œil nu, mais mesurables par machine.

Contrainte prioritaire :

Maximiser la détection des faux billets : un faux non détecté est un faux en circulation.



Le dataset : 1 500 billets scannés



6 features géométriques (en mm) :

- **length** : longueur du billet
- **height_left / height_right** : hauteur gauche et droite
- **margin_up / margin_low** : marges haut et bas
- **diagonal** : diagonale du billet



1 target :

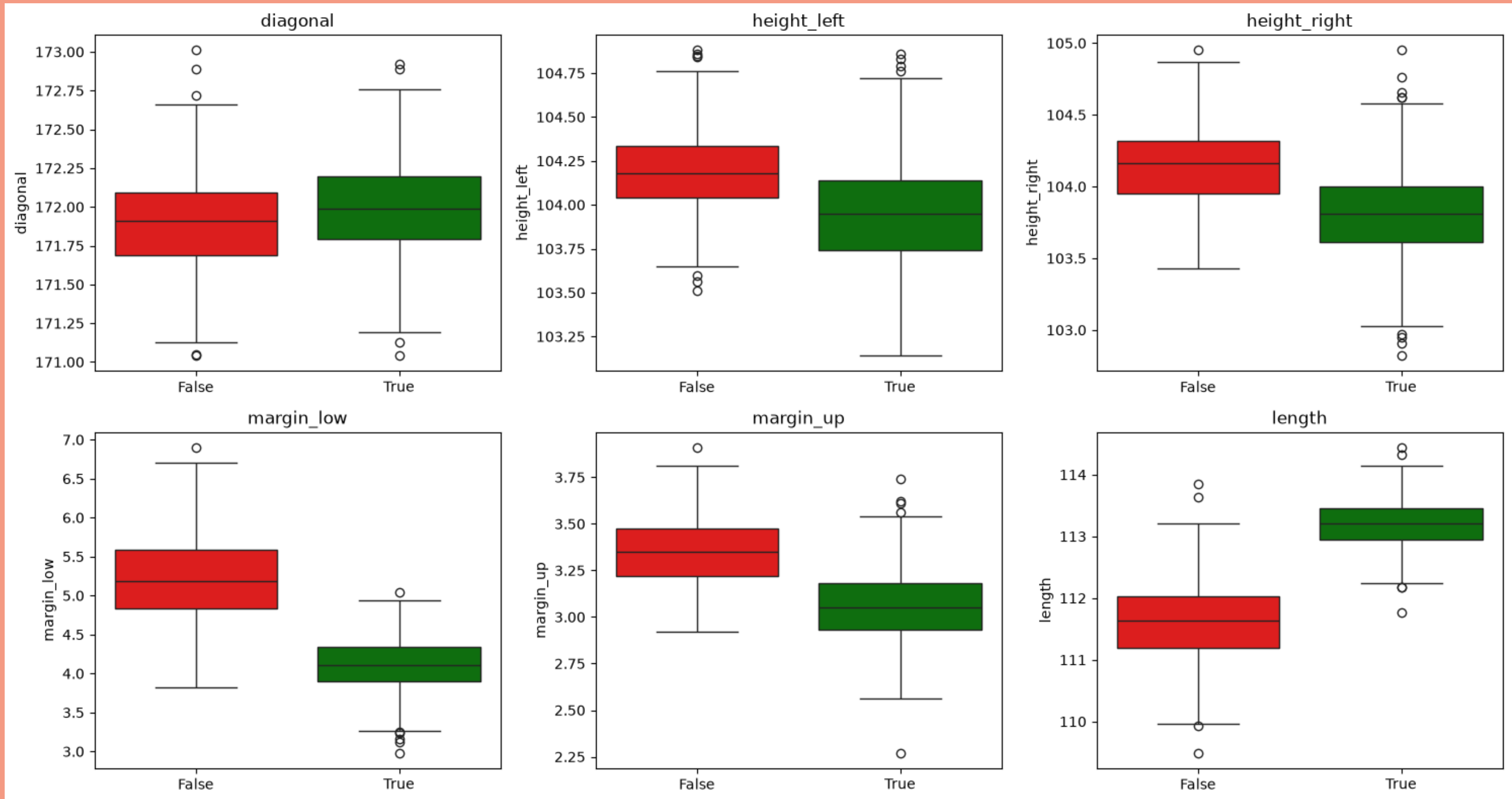
- **is_genuine** : si le billet est vrai (True) ou faux (False)



Répartition des classes :

- 1 000 vrais billets (66,7 %)
 - 500 faux billets (33,3 %)
- Déséquilibre 2:1 à prendre en compte dans la modélisation

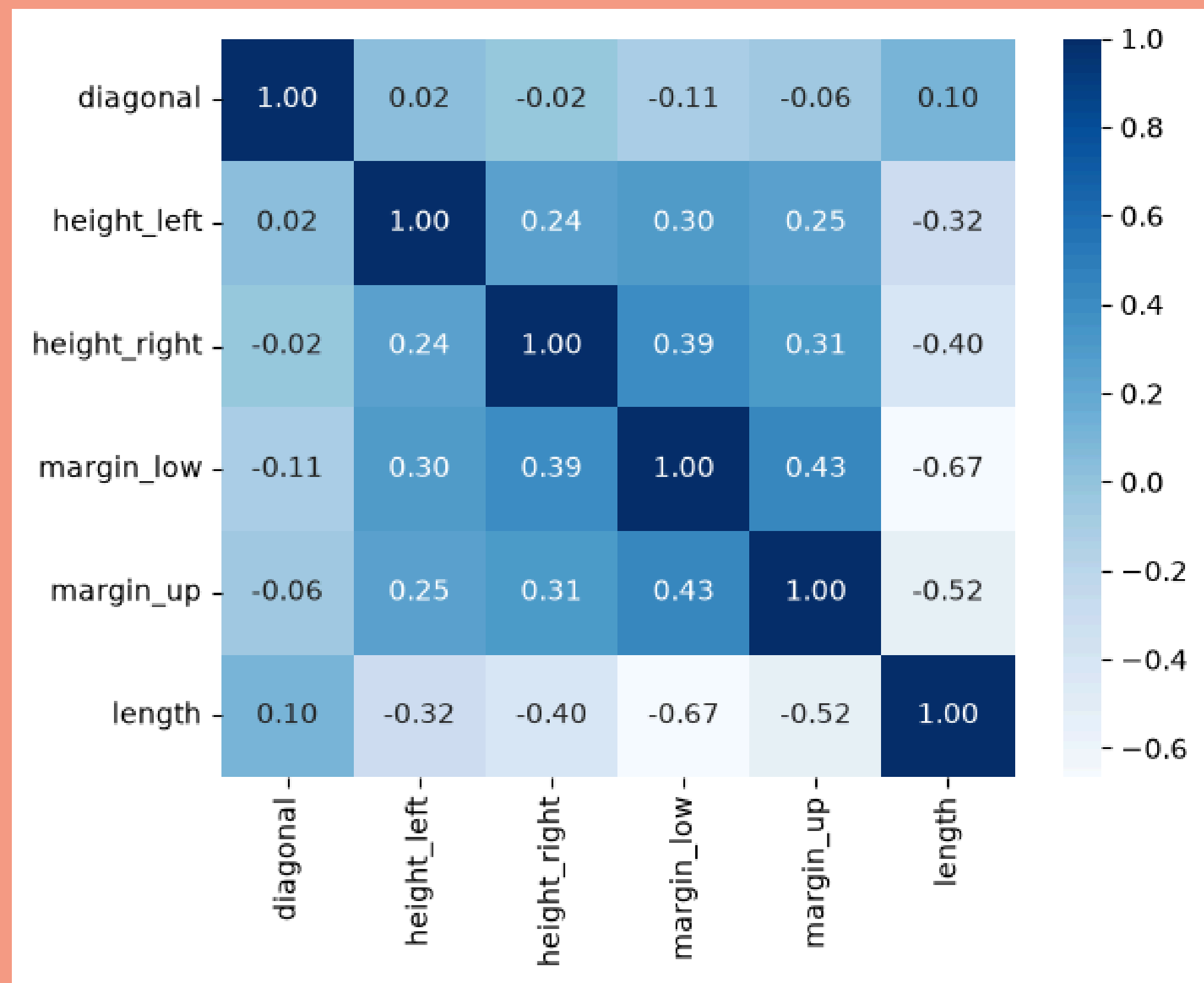
EDA : Distributions



EDA : Corrélations

Corrélation avec is_genuine :

- length : **+0,849** ← **feature n°1**
- margin_low : **-0,786** ← **feature n°2**
- margin_up : -0,606
- height_right : -0,485
- height_left : -0,379
- diagonal : +0,133 ← presque inutile seule



Prétraitement : Split & Imputation



Train / Test Split :

- 80 % entraînement (1 200 billets) / 20 % test (300 billets)
- stratify=y : pour conserver le ratio 2:1 dans chaque ensemble
- random_state=42 : résultats reproductibles



Méthode d'imputation choisie : médiane du groupe TEST

- Plus robuste aux valeurs extrêmes que la moyenne
- Éviter le data leakage

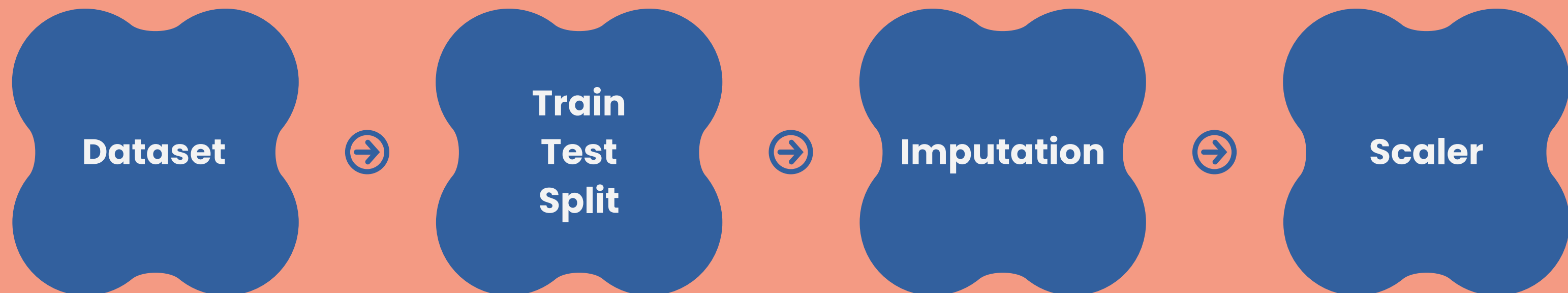
Seule la variable margin_low présente des valeurs manquantes : 37 valeurs (~2,5 % du dataset)

Prétraitement : Standardiser



Standardisation (StandardScaler) :

- Appliqué sur X_train (fit + transform), puis sur X_test (transform uniquement)
 - Règle d'or respectée : jamais de fit sur le test set → éviter le data leakage
 - Nécessaire pour : Régression Logistique, KNN, K-Means
- Non nécessaire pour : Random Forest (insensible à l'échelle des features)



Les 4 algorithmes testés

Algorithme	Type	Caractéristique clé
K-Means	Non supervisé	Clustering (sans labels)
Régression Logistique	Supervisé	Probabilités (interprétable)
KNN	Supervisé	Proximité entre voisins
Random Forest	Supervisé	Ensemble d'arbres de décision



Métriques d'évaluation :

- Recall Faux ← métrique prioritaire (détecter les contrefaçons)
- Accuracy, AUC-ROC, matrice de confusion

K-Means



Principe : Regrouper les billets en 2 clusters sans jamais voir les labels → les centroïdes émergent naturellement des données.



Bilan : Performance remarquable pour un algorithme n'ayant jamais eu accès aux labels → confirme que les différences géométriques sont réelles et structurées.



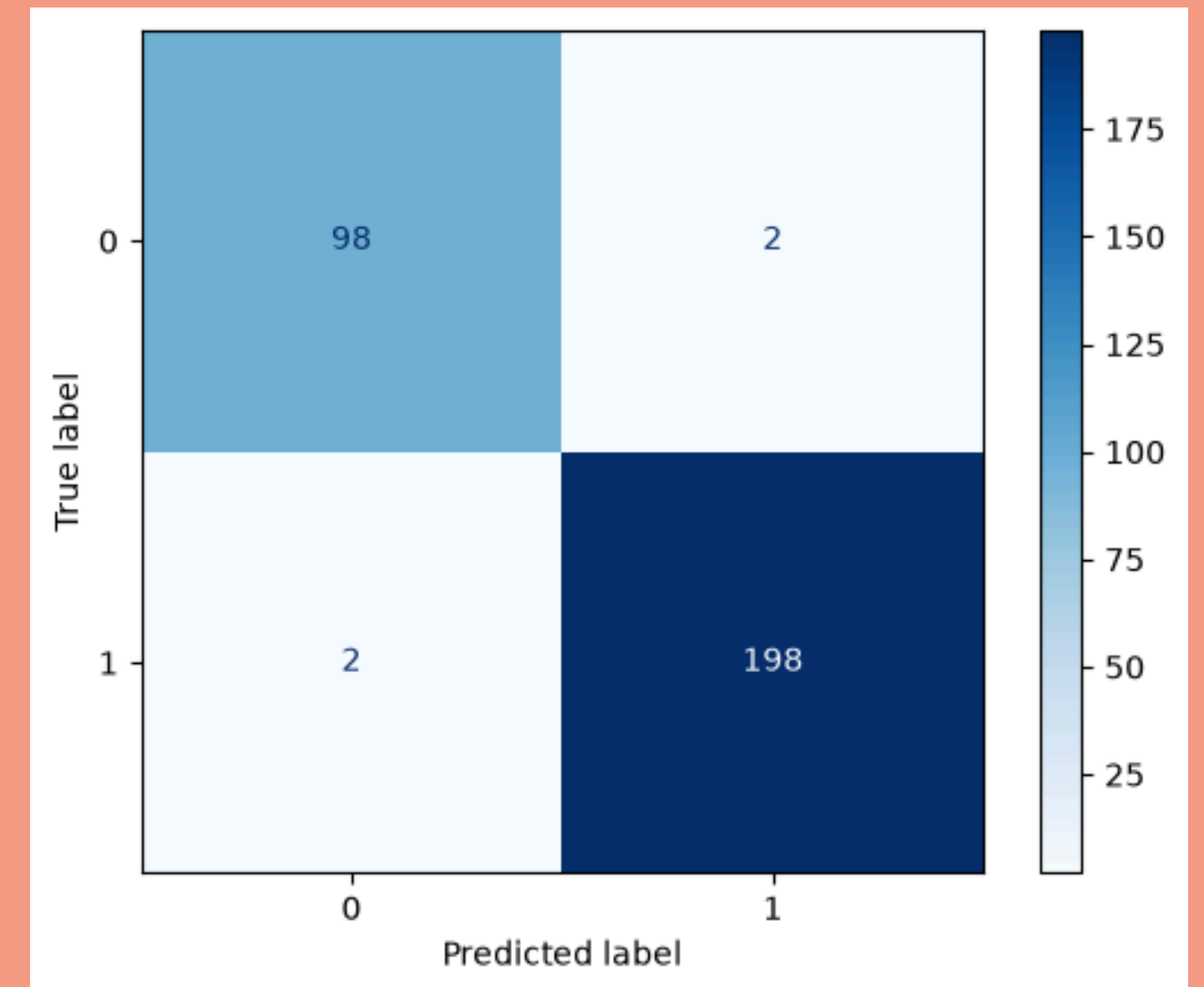
Résultats :

- Accuracy : **0,99**
- Recall Faux : **0,98**



Particularité :

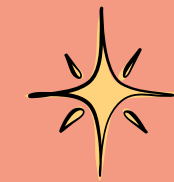
Attribution manuelle des clusters (Vrai/Faux) après entraînement, via les taux de « genuinité » de chaque groupe.



RÉgression Logistique



Principe : Modélise la probabilité qu'un billet soit faux. Un seuil de décision transforme cette probabilité en classe binaire.

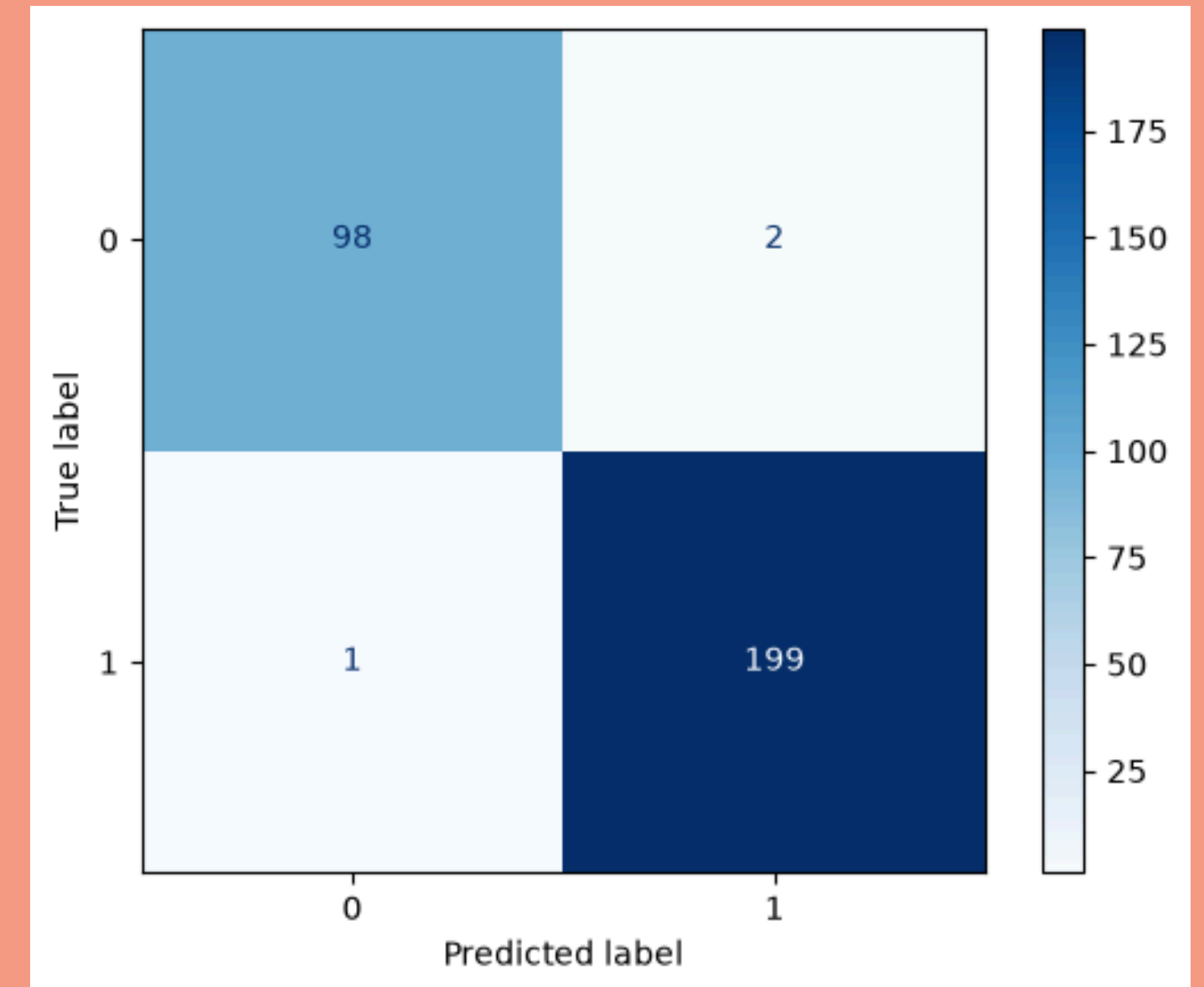


Avantages : Interprétable, rapide, probabilités calibrées.



Résultats :

- Accuracy : **0,99**
- AUC : **0,9995** ← meilleur de tous les modèles
- Recall Faux : **0,98**



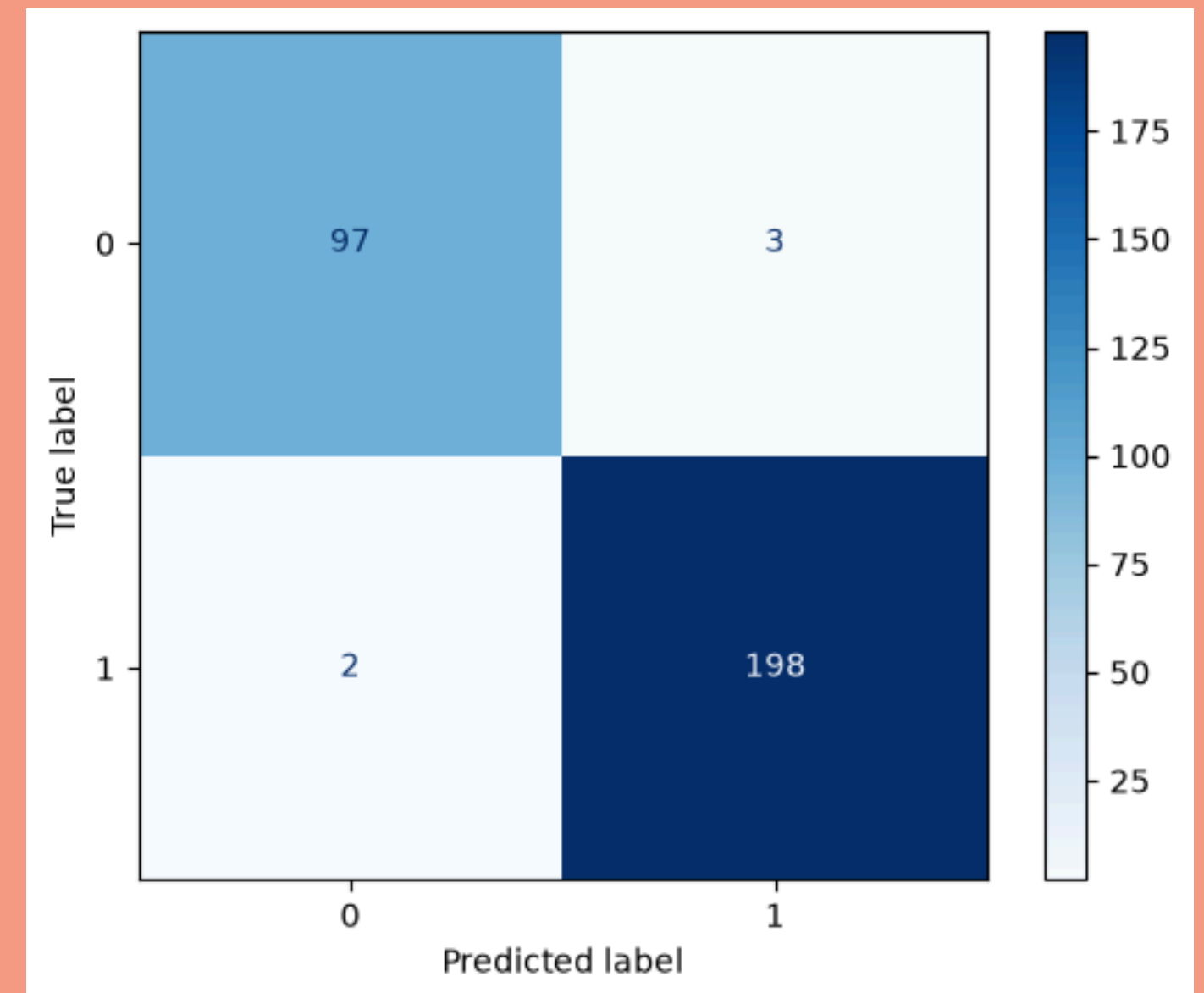
KNN

✦ **Principe** : Un billet est classifié selon la majorité de ses k voisins les plus proches dans l'espace des features.

✦ **Bilan** : Modèle solide mais systématiquement le moins performant sur ce dataset.

✦ **Résultats (k=5)** :

- Accuracy : **0,98**
- AUC : **0,9971**
- Recall Faux : **0,97** ← le plus faible



Random Forest



Principe : Agrège les prédictions de 100 arbres de décision entraînés sur des sous-ensembles aléatoires → robuste et puissant.



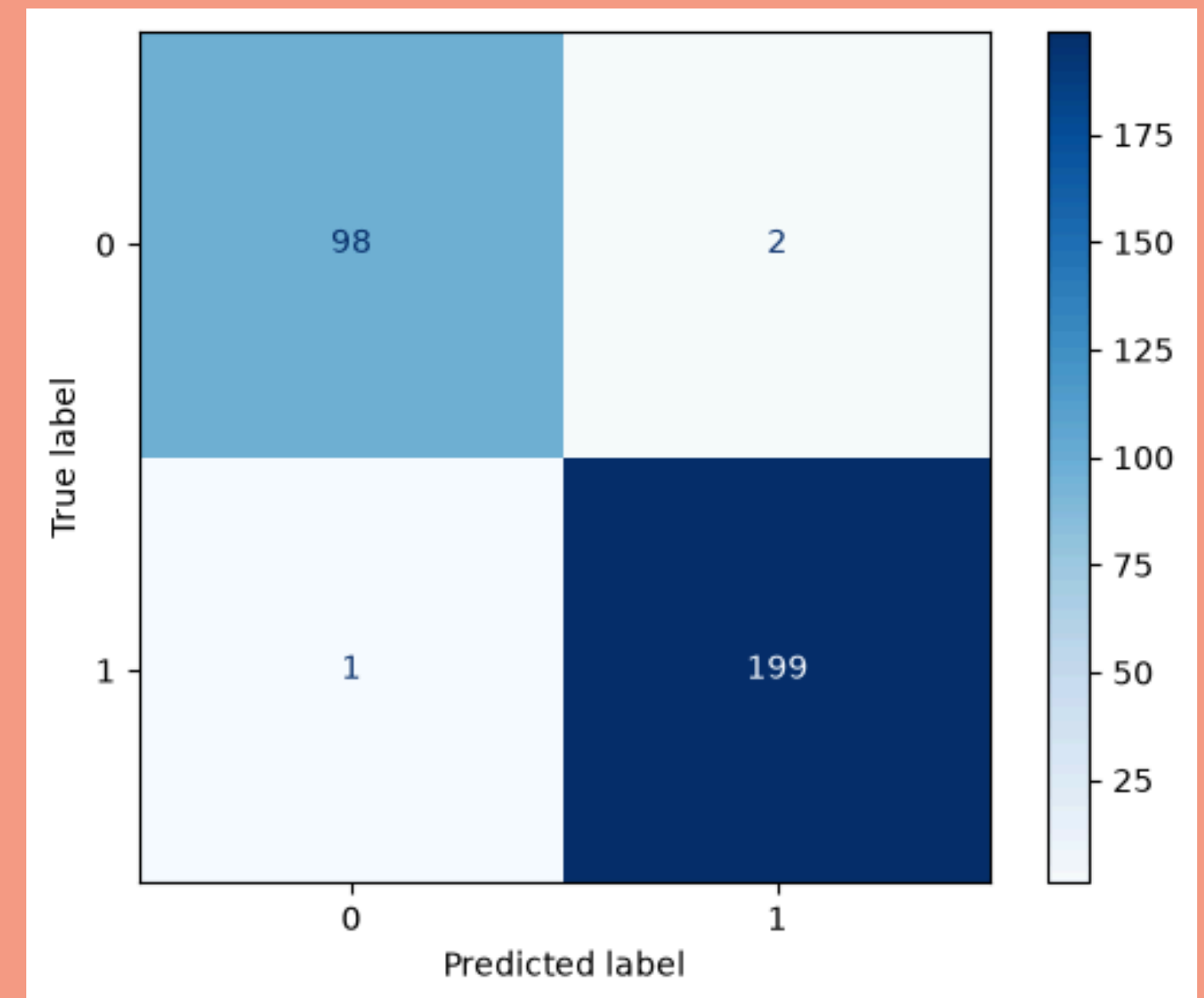
Importance des features (confirmée) :

length et margin_low dominant → cohérent avec l'EDA.



Résultats :

- Accuracy : **0,99**
- AUC : **0,9992**
- Recall Faux : **0,98**



Optimisation des hyperparamètres

✦ **Méthode** : GridSearchCV avec validation croisée à 5 plis (cv=5)

✦ **Métrique d'optimisation** : Recall des faux billets (pos_label=False)

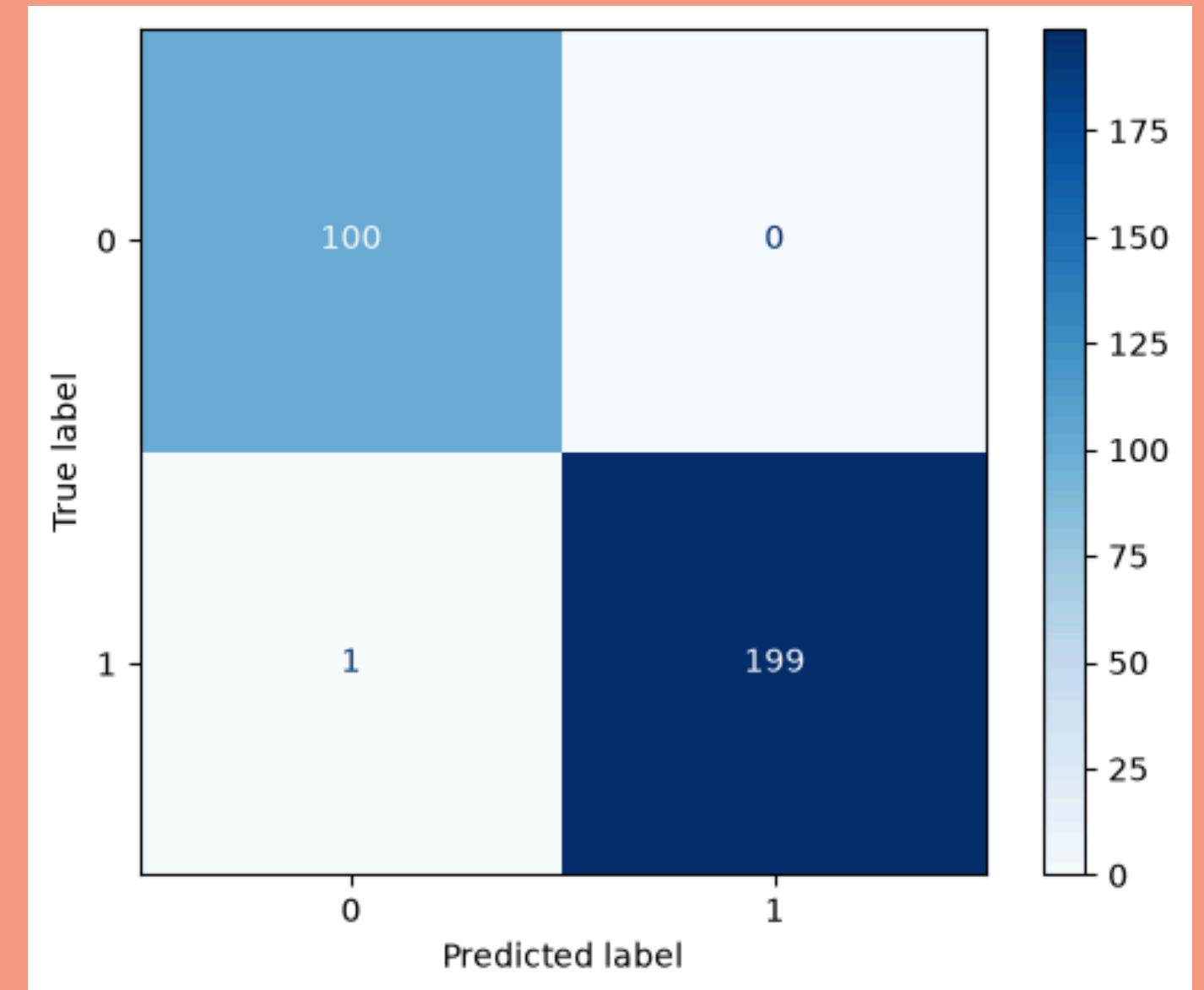
✦ **Régression Logistique paramètres testés** :

- **C** : [0,1, 1, 5, 10]
- **class_weight** : pondération des classes jusqu'à {Faux: 5, Vrai: 1}
- **max_iter** : [1000, 2000, 3000]

✦ **Meilleure configuration trouvée** :
C=1, class_weight={Faux: 5, Vrai: 1}, max_iter=1000

✦ **Résultats** : Recall Faux passe de 0,98 → **1,00**

- Optimisation du seuil de décision (threshold = 0.7636)
→ résultats encore améliorés.



Modèle final retenu



- **Recall Faux = 1,00** → aucun faux billet manqué sur le jeu de test
- **Meilleur AUC (0,9995)** → meilleure capacité de discrimination globale
- **Interprétable** → les équipes peuvent comprendre les décisions
- **Léger et rapide** → adapté à une utilisation en production quotidienne
- **Probabilités calibrées** → chaque prédiction est accompagnée d'un score de confiance



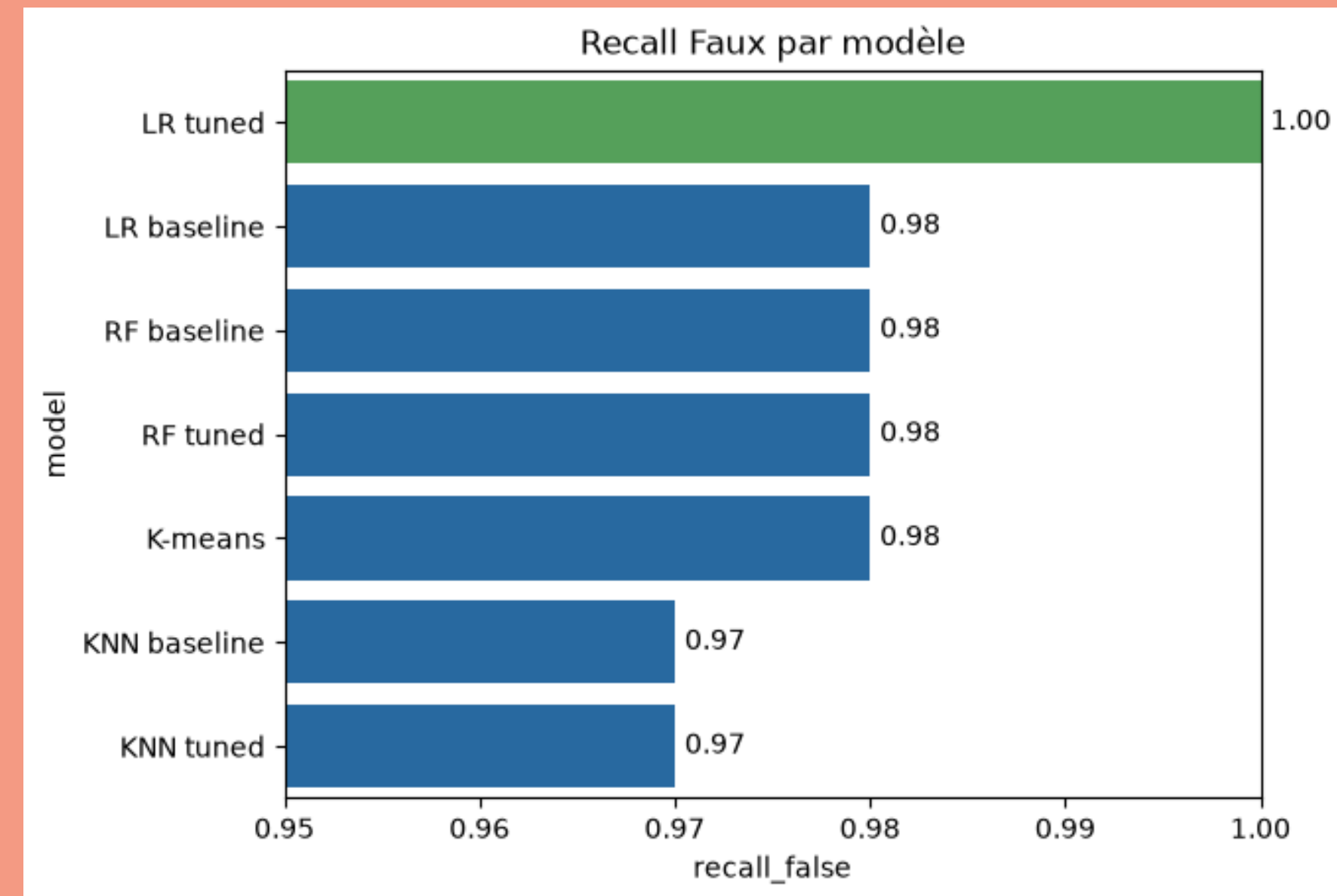
Pipeline final :

Imputation médiane

→ StandardScaler

→ Régression Logistique avec des hyperparamètres

→ Sauvegardé au format .pkl (joblib)



L'application de prédiction



Deux modes d'entrée :

```
# Mode fichier CSV
./script.py --csv billets_production.csv

# Mode valeurs manuelles
./script.py --values 171.9 103.9 104.0 5.0 3.4 113.0
```



Fonctionnalités :

- Validation des colonnes
- Messages d'erreur clairs
- Export CSV automatique des résultats.

id	prediction	probability_of_being_fake
A_1	Fake	0.999
A_2	Fake	0.998
A_3	Real	0.019

Conclusion



Ce qui a été réalisé :

- Prétraitement complet : imputation, split stratifié, standardisation
- Analyse exploratoire approfondie : length et margin_low sont les variables clés
- Test et comparaison de 4 algorithmes (supervisé et non supervisé)
- Optimisation par GridSearchCV + ajustement du seuil de décision
- Pipeline production sauvegardé et script fonctionnel



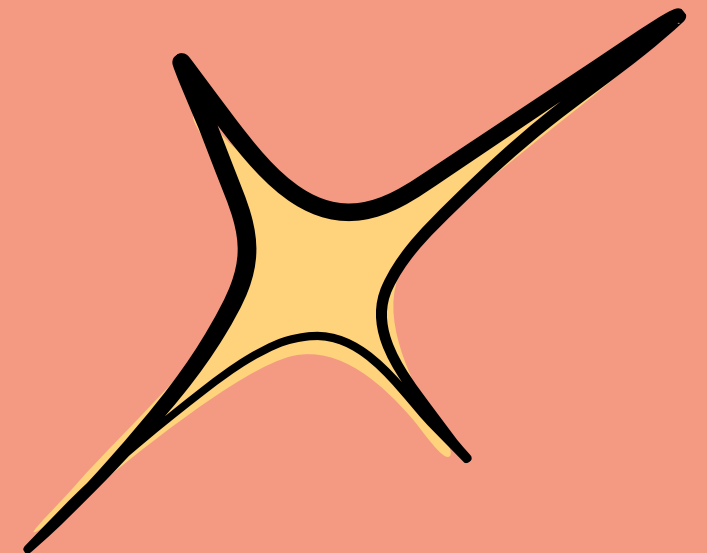
Modèle déployé :

- Régression Logistique avec pondération des classes
→ **0 faux billet manqué sur le jeu de test**

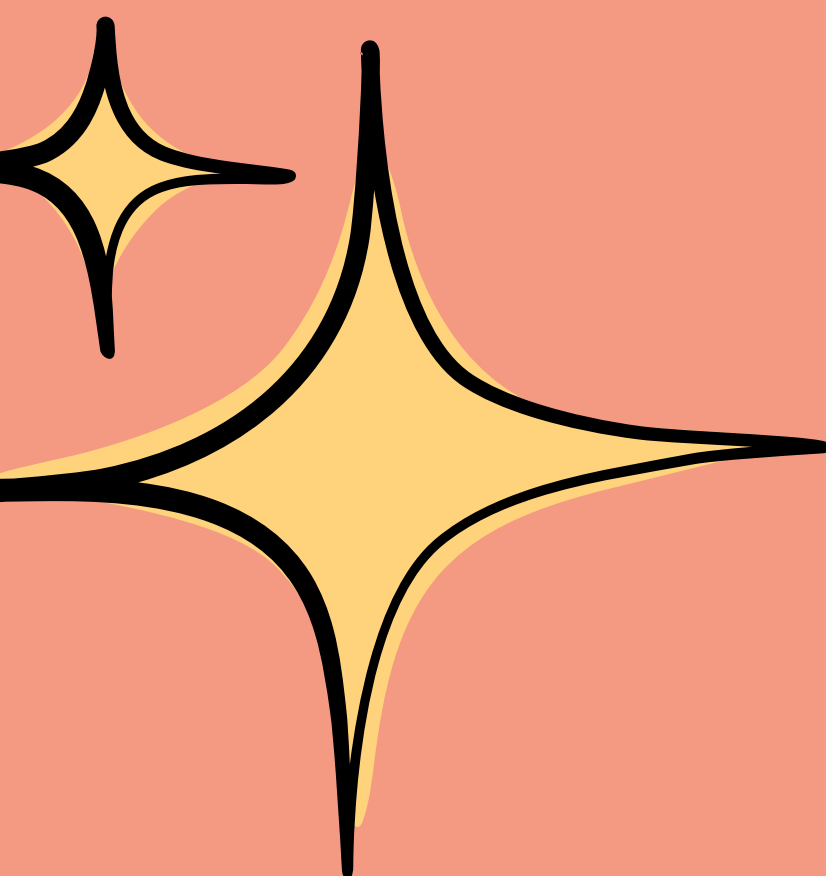


Prochaines étapes possibles :

- Enrichissement du dataset au fil des scans
- Réentraînement périodique du modèle
- Intégration dans le système de scan existant



DÉMONSTRATION en direct



1. Activation de l'environnement virtuel

2. Chargement du fichier fourni

3. Exécution du script

4. Lecture des résultats

